



LOOM QGIS Plugin

User Guide

LOOM QGIS Plugin User Guide v1

Date 8/24/2026

File Name	V	Date	Changes	Authors	Contributors
LOOM QGIS Plugin - User Guide.docx	I	24/08/2026 13:00:00			

Table of Contents

Table of Contents	2
List of Figures	3
List of Tables	3
1. Introduction	4
1.1 Who this guide is for	4
1.2 Tool requirements	4
2. Installing the Plugin	5
2.1 Option 1: Install from QGIS Plugin Repository	5
2.2 Where to find the plugin	5
2.3 Option 2: Manual installation from ZIP	6
2.3 First-run: Binary download	6
3. Plugin Overview and the Pipeline	7
4. Plugin Tabs	8
4.1 Input tab	8
4.2 Topo tab	9
4.3 Loom tab	10
4.3.1 Line ordering (optimisation method)	10
4.3.2 Crossing and separation penalties	11
4.4 Octi tab	12
4.4.1 Schematisation toggle	12
4.4.2 Base graph	13
4.4.3 Grid size	13
4.4.4 Direction and angle penalties	13
4.4.5 ILP options	14
4.5 Render tab	15
4.6 Output tab	18
4.7 Diagnostics tab	20
5. Running the Pipeline	22
6. Recommended Starting Setup	22
7. Where LOOM Fits in the TfC Workflow	22
8. Output examples	24
9. External references	26

List of Figures

Figure 1 Installing the plugin – select the Plugin menu	5
Figure 2 LOOM Transit Map Generator in the Plugins menu.....	5
Figure 3 LOOM Transit Map Generator in the Plugins drop-down menu.....	5
Figure 4 Installing the plugin - Insert from ZIP.....	6
Figure 5 LOOM QGIS Plugin – Input tab user interface	8
Figure 6 LOOM QGIS Plugin – Topo tab user interface.....	10
Figure 7 LOOM QGIS Plugin – Loom tab user interface	12
Figure 8 LOOM QGIS Plugin – Octi tab user interface.....	15
Figure 9 LOOM QGIS Plugin – Render tab user interface	17
Figure 10 LOOM QGIS Plugin – Output tab user interface.....	19
Figure 11 LOOM QGIS Plugin – Diagnostics tab user interface.....	21
Figure 12 LOOM plugin position in the TfC Napping Workflow (Transit Map phase).....	23
Figure 13 svg output example using GTFS for Cairo Metro lines 1, 2, and 3, ortholinear schematisation, with random colours	24
Figure 14 Web map output example using GTFS for public transport in Tunisia, no schematisation, with random colours	24
Figure 15 Web map output example using GTFS for popular transport in Tanzania, orthoradial schematisation with random colours	25

List of Tables

Table 1 LOOM pipeline stages	7
Table 2 Input tab options.....	8
Table 3 Topo tab options	9
Table 4 Optimisation method options.....	10
Table 5 Crossing and separation penalty fields.....	11
Table 6 Base graph styles.....	13
Table 7 Direction penalty fields.....	13
Table 8 Angle penalty fields.....	13
Table 9 Render tab options.....	16
Table 10 Recommended starting setup	22

I. Introduction

The LOOM Transit Map Generator is a QGIS plugin developed by Transport for Cairo (TfC) that brings the LOOM suite into a point-and-click interface, making schematic and geographic transit map production accessible directly inside QGIS — with no command-line required.

[LOOM](#) (Line-Ordering-Optimization-Model) is an open-source suite developed by Hannah Bast, Patrick Brosi, and Sabine Storandt at the University of Freiburg. It automates the generation of high-quality transit maps from GTFS or GeoJSON data by solving the problem of optimal line ordering along shared road corridors. The plugin wraps the same LOOM binaries in a friendly Qt dialog, and pre-built binaries are downloaded automatically on first run — no compilation required.

I.1 Who this guide is for

This guide is intended for:

- Transport analysts and cartographers working with GTFS or GeoJSON data who want to produce publication-quality transit maps inside QGIS.
- Researchers and planners who use the TfC data workflow and want to generate transit maps as a downstream step after producing a validated GTFS feed.
- Developers or contributors who wish to understand or extend the plugin.

The plugin can be used independently with any GTFS feed or GeoJSON line layer, or as the final mapping step in the TfC workflow after the GIS2GTFS tool has produced a validated GTFS output.

I.2 Tool requirements

The plugin is currently compatible with QGIS 3.x and later, and runs on Windows, macOS, and Linux. An internet connection is required on first run to download the pre-built LOOM binaries (approximately 10–80 MB, one time only).

2. Installing the Plugin

2.1 Option 1: Install from QGIS Plugin Repository

LOOM QGIS Plugin is available directly through QGIS' Plugin Repository:

1. Download and open [QGIS](#) (version 3.x or higher)
2. From the menu bar, go to **Plugins → Manage and Install Plugins (Error! Reference source not found.)**
3. Search for **LOOM Transit Map Generator**.
4. Click **Install Plugin**, and make sure it's checked to enable it as in Figure 2.

2.2 Where to find the plugin

After installation:

You can find LOOM under **Plugins → LOOM Transit Maps → LOOM Transit Map Generator** as in Figure 3.

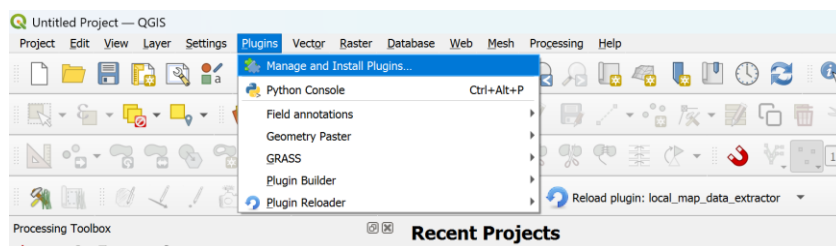


Figure 1 Installing the plugin – select the Plugin menu

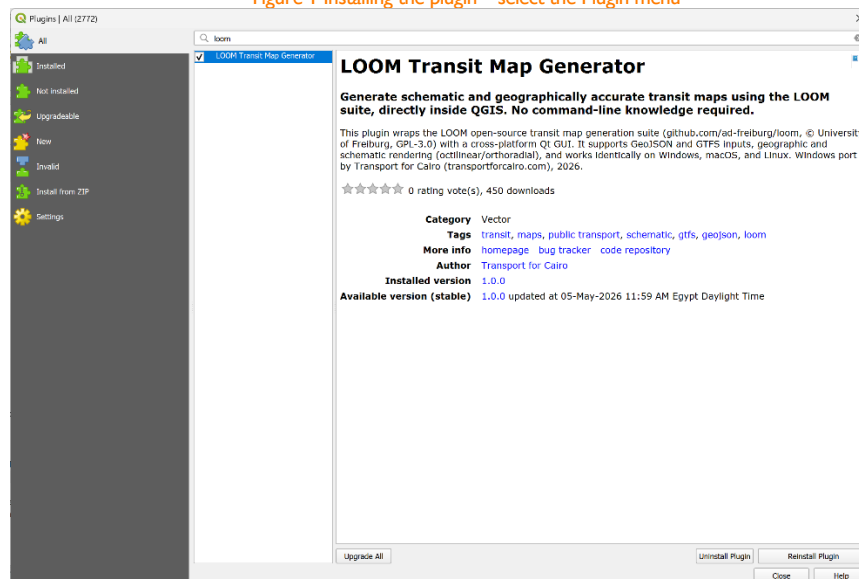


Figure 2 LOOM Transit Map Generator in the Plugins menu

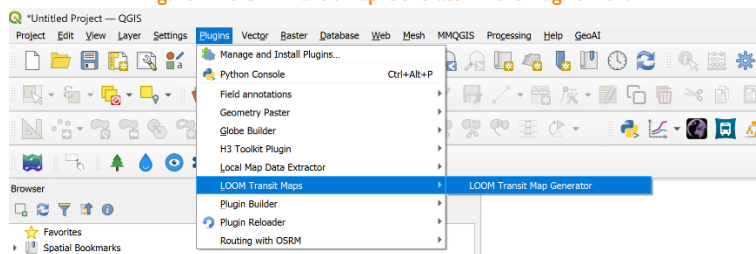


Figure 3 LOOM Transit Map Generator in the Plugins drop-down menu

I.3 Option 2: Manual installation from ZIP

1. Download the latest plugin ZIP from [the GitHub repository](#)
2. Download and open [QGIS](#) (version 3.x or higher)
3. From the menu bar, go to **Plugins → Manage and Install Plugins (Error! Reference source not found.)**
4. In the dialog that appears, click the **Install from ZIP** → click **Browse** and select the downloaded ZIP file on your PC → click **Install Plugin** → click **Close** (Figure 4)

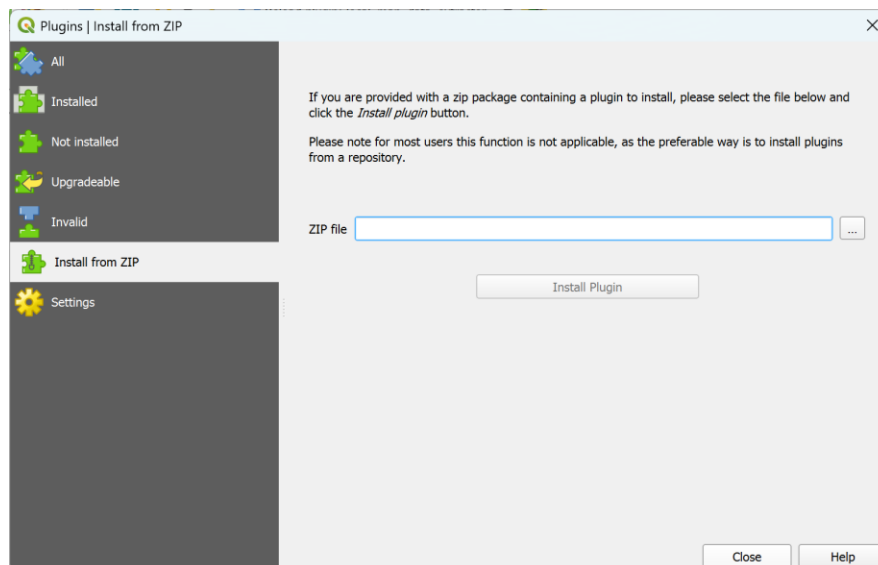


Figure 4 Installing the plugin - Insert from ZIP

2.3 First-run: Binary download

On the very first launch, the plugin detects that the LOOM binaries are not yet present and opens an automatic download dialog. The dialog downloads the pre-built binaries for your platform (Windows, macOS, or Linux) from the TfC binary repository and installs them into the plugin folder. The download is approximately 10–80 MB and only needs to happen once. Subsequent launches proceed directly to the main dialog.

3. Plugin Overview and the Pipeline

The LOOM Transit Map Generator wraps five command-line tools from the LOOM suite in a single QGIS dialog. When the user clicks Run Pipeline, the plugin chains these tools in sequence, passing the output of each stage as the input of the next. The table below summarises the pipeline.

Table 1 LOOM pipeline stages

Stage	Tool	What it does
1	gtfs2graph	Converts a GTFS zip into a GeoJSON line graph. Fires only when a GTFS zip is provided as input.
2	topo	Resolves overlapping edges and merges nearby parallel corridors into shared segments, producing a topologically valid planar graph.
3	loom	Optimises the ordering of lines within each shared corridor to minimise visual crossings in the final map.
4	octi (optional)	Schematises the map by forcing line segments onto a geometric grid (octilinear, orthoradial, etc.). Disabled by default; omitting it produces a geographically accurate map.
5	transitmap	Renders the final output as an SVG file (for Illustrator/Inkscape) or as MVT vector tiles (for web maps).

Note: Stage 4 (octi) is optional. When the Run octi checkbox on the Octi tab is unchecked, the pipeline skips schematisation and produces a map whose lines follow their real geographic coordinates. Enable octi only when a schematic transit-map style is required.

4. Plugin Tabs

4.1 Input tab

The Input tab defines the data source for the pipeline. Exactly one of the three input options should be populated; if more than one is filled in, the GTFS zip takes priority, followed by the GeoJSON file, and finally the QGIS layer.

Table 2 Input tab options

Field	Description
QGIS layer	A vector line layer already loaded in the current QGIS project. Select it from the drop-down. The layer is exported to GeoJSON internally before being passed to the pipeline.
GeoJSON file	A .geojson file on disk. Click Browse to locate it. Use this when the input data is not already loaded as a QGIS layer.
GTFS zip	A validated GTFS .zip file. When a GTFS zip is provided, the pipeline fires the gtfs2graph stage first to convert it into a line graph before continuing. The GIS2GTFS plugin in TfC Tools can be used to produce this file.

Figure 5 LOOM QGIS Plugin – Input tab user interface

4.2 Topo tab

The Topo tab controls the topo stage, which resolves the raw line graph geometry so that lines only intersect at designated station nodes. When two lines cross between stations, topo inserts a virtual node at the intersection, turning the geometric tangle into a valid planar graph. The Run topo stage checkbox should remain enabled in almost all cases.

Table 3 Topo tab options

Field	Description
Run topo stage	Checked by default. Resolves crossings between stations and merges nearby parallel segments. It is strongly recommended to leave this enabled; disabling it may cause downstream errors in the loom and transitmap stages.
Max aggr. dist (m)	Default 50 m. Distance below which two parallel segments are merged into one shared corridor. Increase this value if parallel lines that should be bundled still appear as separate lines in the output.
Max comp. dist (m)	Default 10 000 m. Maximum distance between two nodes for them to be considered part of the same connected component. Reduce for compact city-centre maps; increase for regional networks.
--smooth	Smooths the output edge geometries. Enable if the raw line shapes appear jagged after topo processing.
--random-colors	Assigns random hex colours to routes that have no colour defined in the GTFS data. Enable if the output map renders all lines in the same default colour.
--no-infer-restrs	Disables automatic turn-restriction inference. Enable this option if routes appear to be cut off at junctions unexpectedly.
--write-stats	Writes processing statistics to the output. Intended for diagnostic use.
--write-components	Tags each edge with a component ID. Useful for identifying disconnected sub-networks in complex datasets.

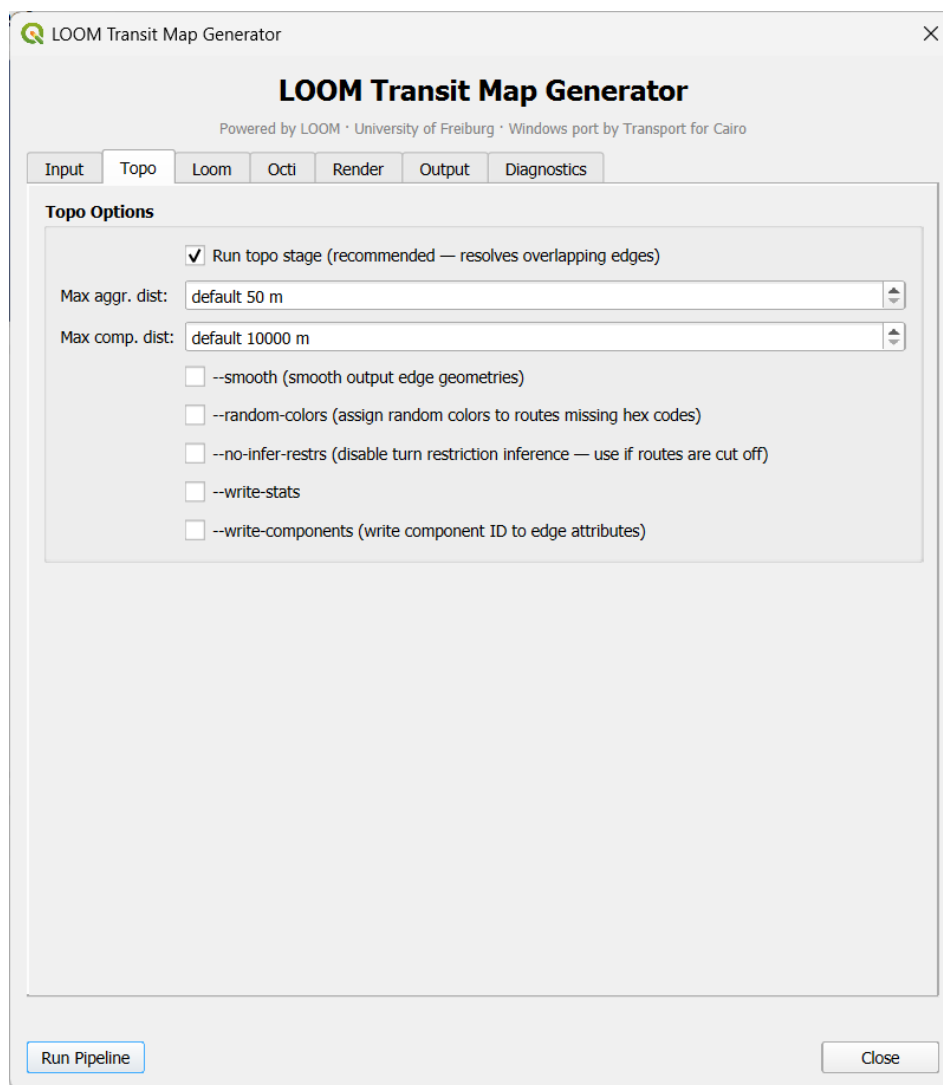


Figure 6 LOOM QGIS Plugin – Topo tab user interface

4.3 Loom tab

4.3.1 Line ordering (optimisation method)

The loom stage determines the stacking order of lines within each shared corridor so that the final map has the fewest possible crossings. The optimisation method drop-down controls the algorithm used. The table below describes each option.

Table 4 Optimisation method options

Method	Description
comb-no-ilp	Default. Fast and high quality. Does not require an ILP solver. Recommended starting point for all networks.
comb	Best visual quality. Uses an ILP solver for the final step. Use when output quality matters and a solver is available.
ilp	Mathematically optimal. Very slow. Only suitable for small networks (fewer than approximately 5 routes).
ilp-naive	Simpler ILP formulation. Sometimes faster than ilp on certain network topologies.

greedy	Very fast, lower quality. Suitable for quick previews.
greedy-lookahead	Greedy with a one-step lookahead. Slightly better than greedy.
hillc / hillc-random	Hill climbing. Suitable for medium-sized networks.
anneal / anneal-random	Simulated annealing. Suitable for medium-sized networks where greedy quality is insufficient.
exhaust	Exhaustive search. Tries every possible line-order combination. Only for very small corridors (two to four lines). Will not complete on larger networks.
null	No optimisation applied. Use only for diagnostic comparison.

The `--no-untangle` checkbox disables the untangling pass that resolves knotted line orderings; enable it only if the tool is over-simplifying your network. The `--no-prune` checkbox disables the pruning of short stub segments; enable it if important short line segments are disappearing from the output.

4.3.2 Crossing and separation penalties

The penalty fields control how strongly the optimiser avoids crossings and gaps. All fields default to zero, which instructs the plugin to use LOOM's built-in default values shown in the table below. Only adjust these values if specific crossing artefacts appear in the output and cannot be resolved by switching optimisation method.

Table 5 Crossing and separation penalty fields

Field	Default	Description
Same-seg cross	4	Penalty for two lines crossing within the same corridor segment.
Diff-seg cross	1	Penalty for two lines crossing across different corridor segments.
Station cross (same seg)	12	Penalty for lines crossing at a station within the same segment.
Station cross (diff seg)	3	Penalty for lines crossing at a station across different segments.
Sep pen	3	Penalty for unnecessary gaps between lines in a corridor.
Station sep pen	9	Penalty for unnecessary gaps between lines at a station.


LOOM Transit Map Generator

LOOM Transit Map Generator

Powered by LOOM · University of Freiburg · Windows port by Transport for Cairo

Input
Topo
Loom
Octi
Render
Output
Diagnostics

Line Ordering

Optim method: comb-no-ilp

☐ --no-untangle

☐ --no-prune

Crossing Separation Penalties (leave at 0 to use defaults)

Same-seg cross: default 4

Diff-seg cross: default 1

Station cross (same): default 12

Station cross (diff): default 3

Sep pen: default 3

Station sep pen: default 9

ILP Options (only apply to ilp / comb methods)

ILP solver: auto

Time limit: unlimited

Threads: solver default

Run Pipeline
Close

Figure 7 LOOM QGIS Plugin – Loom tab user interface

4.4 Octi tab

4.4.1 Schematisation toggle

The Run octi checkbox at the top of the tab enables or disables the schematisation stage. When unchecked, the pipeline produces a geographically accurate map whose lines follow their real-world

coordinates. When checked, the octi tool forces lines onto a geometric grid, producing the schematic transit-map style. Disable octi for a first run and enable it once the geographic output has been validated.

4.4.2 Base graph

The Base graph drop-down selects the grid style used for schematisation. The table below describes each option.

Table 6 Base graph styles

Style	Description
octilinear	Eight-directional grid. Produces the classic metro-map appearance with lines at 0°, 45°, and 90° angles. Recommended default for schematic maps.
ortholinear	Right-angle only. Forces all lines to 0° and 90°. Suitable for circuit-board style maps.
orthoradial	Concentric circles and radial spokes. Use for maps centred around a specific terminal or city core.
quadtree	Adaptive quadtree grid. Adjusts cell size based on local network density.
octihanan	Hanan grid. Minimises total line length. Use for very dense urban networks.

4.4.3 Grid size

The Grid size field accepts either a percentage of the average station-to-station distance (e.g. 120%) or an absolute value in metres (e.g. 80). Lower values create finer grids with more detail; higher values produce more abstract, coarser maps. Leave blank to use the LOOM default of 100%.

4.4.4 Direction and angle penalties

All direction and angle penalty fields default to zero, which uses LOOM's built-in defaults listed below. These fields are provided for advanced cartographic fine-tuning and do not need to be changed for standard use.

Table 7 Direction penalty fields

Field	Default	Description
Geo pen	0	Penalty for moving lines away from their real-world coordinates. Set above 0 to keep the schematic map geographically faithful.
Diag pen	0.5	Penalty for using diagonal (45°) edges.
Vert pen	0	Penalty for vertical edges.
Hori pen	0	Penalty for horizontal edges.
Density pen	10	Penalty for placing lines in already-crowded areas of the grid.

Table 8 Angle penalty fields

Field	Default	Description
Pen 90°	1.5	Penalty for sharp right-angle bends.
Pen 45°	2	Penalty for 45° bends.
Pen 135°	1	Penalty for shallow 135° bends.
Pen 180°	0	Penalty for U-turns.

Nd move pen	0.5	Penalty for moving a station node away from its original position.
-------------	-----	--

4.4.5 ILP options

The ILP time limit field sets a maximum number of seconds for the ILP solver when the octi optimisation mode is set to ilp. The default is 60 seconds. Set to -1 to allow unlimited time. This field has no effect when the optimisation mode is set to heur.



4.5 Render tab

The Render tab controls the visual appearance of the output map. The table below describes all available options; fields left at their special-value text (shown as 'default N' in the spin box) will use LOOM's built-in defaults.

Table 9 Render tab options

Field	Default	Description
Output format	svg	SVG (recommended) produces a vector file suitable for Illustrator or Inkscape. MVT produces vector tiles for web-map use and requires an MVT output path to be set on the Output tab.
Render labels (-l)	On	Renders station names on the map. Disable to produce a labels-free base layer.
Line width	20	Controls the stroke thickness of transit lines. Increase for larger-format outputs.
Line spacing	10	Sets the gap between parallel lines sharing a corridor.
Outline width	1	Sets the white outline drawn around each line to improve legibility.
Line label size	40	Font size of route labels (the route number or short name displayed on the line itself).
Station label size	60	Font size of station name labels.
Smoothing	1	Applies curve smoothing to rendered line geometry. Higher values produce rounder bends.
Padding	auto	Canvas padding around the map. Leave at -1 for automatic padding.
--no-deg2-labels	Off	Hides station name labels at degree-2 intermediate stops (stops with exactly one incoming and one outgoing line). Enable to reduce label clutter on dense networks.
--render-dir-markers	Off	Adds small direction arrows to lines. Useful when outbound and inbound routes follow different alignments.
--random-colors	Off	Assigns random colours to routes without a defined hex colour. Enable if routes appear in a default grey.
--tight-stations	Off	Prevents the renderer from expanding the station node front. Enable for a more compact station symbol.
--no-render-stations	Off	Suppresses station symbols entirely. Enable when only the line geometry is needed.

LOOM Transit Map Generator

LOOM Transit Map Generator

Powered by LOOM · University of Freiburg · Windows port by Transport for Cairo

Input
Topo
Loom
Octi
Render
Output
Diagnostics

Rendering Options

Output format:
svg

☒ Render labels (-l)

Line width:
default 20

Line spacing:
default 10

Outline width:
default 1

Line label size:
default 40

Station label size:
default 60

Smoothing:
default 1

Padding:
auto (-1)

☐ --no-deg2-labels (hide labels at minor intermediate stops)

☐ --render-dir-markers (show direction arrows on lines)

☐ --random-colors (assign random colors to uncoloured routes)

☐ --tight-stations (don't expand node fronts at stations)

☐ --no-render-stations

Run Pipeline
Close

Figure 9 LOOM QGIS Plugin – Render tab user interface

4.6 Output tab

The Output tab sets where the final output is saved and how it is handled after generation.

The Save SVG to field accepts a file path for the SVG output. Leave it blank to load the result directly into QGIS as a temporary layer without saving to disk. The MVT path and MVT zoom fields are only required when the output format on the Render tab is set to MVT. The zoom field accepts a single zoom level (e.g. 14), a comma-separated list (e.g. 12,13,14), or a range (e.g. 12-15).

The Open SVG in system viewer checkbox automatically opens the generated SVG file in the system default viewer (typically a web browser or graphics editor) as soon as the pipeline completes. This is useful for a quick visual check before the file is imported into Adobe Illustrator or Inkscape for final cartographic editing.


LOOM Transit Map Generator

LOOM Transit Map Generator

Powered by LOOM · University of Freiburg · Windows port by Transport for Cairo

Input

Topo

Loom

Octi

Render

Output

Diagnostics

Output

Save SVG to:

Browse

MVT path (--mvt-path):

Browse

MVT zoom (-z):

☒ Open SVG in system viewer after generation

☐ Generate interactive webmap (HTML)

Webmap HTML:

Browse

☒ Open webmap in browser after generation

SVG Preview

SVG output will appear here...

Run Pipeline

Close

Figure 10 LOOM QGIS Plugin – Output tab user interface

4.7 Diagnostics tab

The Diagnostics tab provides status information about the plugin environment and the LOOM binaries. It is the first place to check when a pipeline run fails.

The Platform section shows the operating system, architecture, and version string detected at runtime. The Binary status section lists each of the six LOOM executables (loom, topo, octi, gtfs2graph, transitmap, topoeval) and indicates whether each was found in the plugin's bin directory or on the system PATH. The Re-check button refreshes this list without restarting the plugin, and the Re-download binaries button triggers a fresh download from the TfC binary repository.

The Last run stderr / warnings panel displays any warning or error messages produced by the LOOM tools during the most recent pipeline run. When a run fails or produces unexpected output, the messages here will typically identify which stage encountered a problem and why.

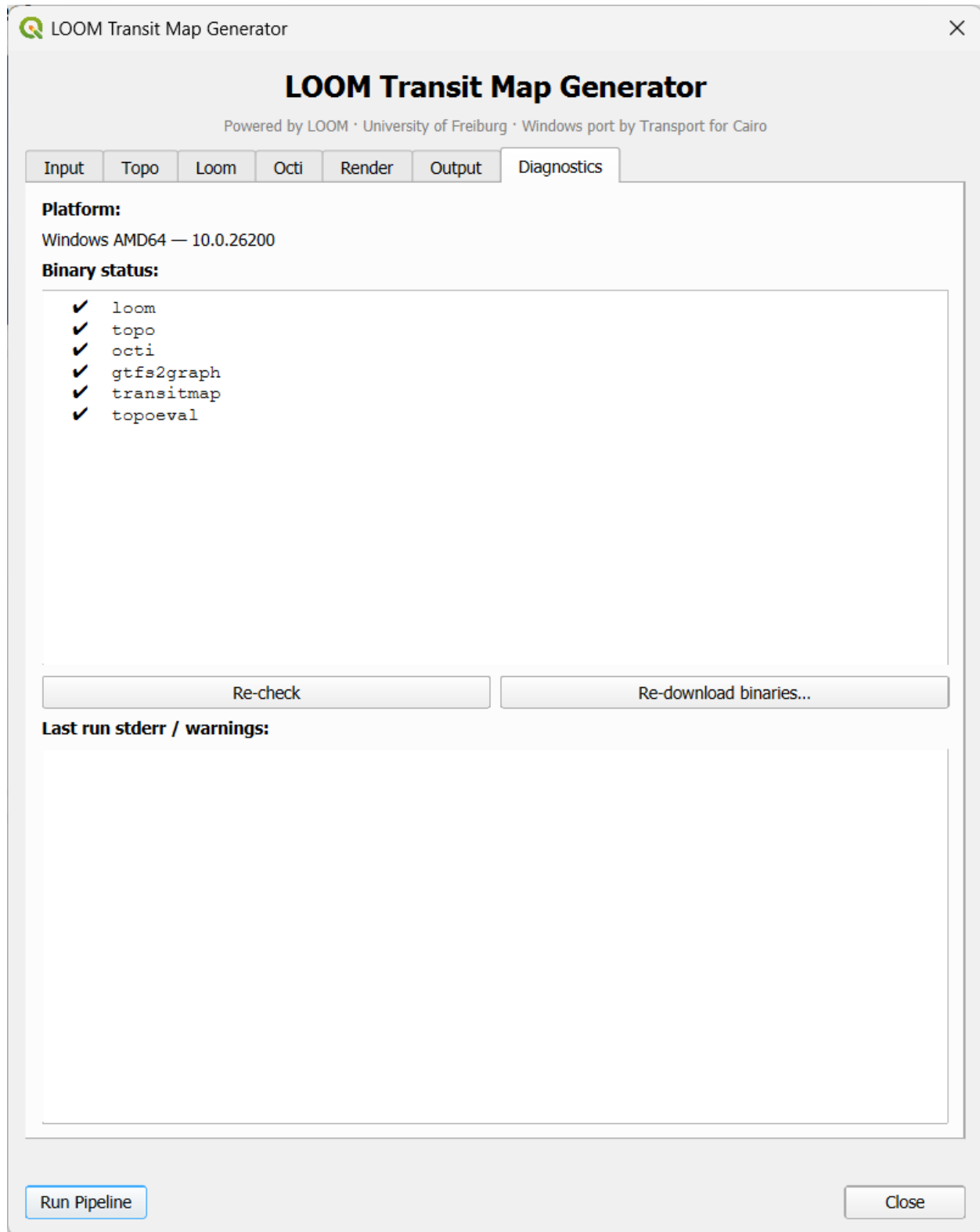


Figure 11 LOOM QGIS Plugin – Diagnostics tab user interface

5. Running the Pipeline

Once the input and any required options have been configured, click Run Pipeline at the bottom of the dialog. A progress bar appears below the tabs and updates as each stage completes. The label next to the bar indicates which stage is currently running.

If the run needs to be stopped, click the Cancel button that appears next to the progress bar. The plugin will terminate the active process cleanly. When the pipeline completes successfully, the SVG output is loaded into the QGIS canvas as a vector layer (if no save path was provided) or saved to disk (if a path was specified), and optionally opened in the system viewer.

If the run fails, an error message will appear in the dialog and additional detail is available in the Diagnostics tab under Last run stderr / warnings.

6. Recommended Starting Setup

The table below provides a recommended starting configuration for new users. It is designed to produce a clean, readable first output with minimal configuration. Once the geographic map has been reviewed and validated, the Octi stage can be enabled and further visual refinements applied through the Render tab.

Table 10 Recommended starting setup

Setting	Recommended value
Input	Any valid GTFS feed in a ZIP format
Topo	Enabled; <i>random-colours</i> checked; all other topo options at their defaults
Loom	<i>comb-no-ilp</i> ; all penalty fields left at 0 (uses LOOM defaults)
Octi	Disabled for a first run to review the geographic map; enable and set to <i>octilinear</i> for a schematic version
Render	<i>Render labels</i> on; <i>random-colours</i> checked; line width 30; line spacing 5; smoothing 3; all other fields at their defaults
Output	Set a save path for the SVG; Open SVG in system viewer checked; Set a save path for the Webmap HTML; Open webmap in browser after generation checked

7. Where LOOM Fits in the TfC Workflow

Within the TfC mapping workflow, the LOOM plugin sits at the Transit Map phase. It receives the Final GTFS feed produced by the GIS2GTFS tool (described in the [TfC Tools Plugin User Guide](#)) and outputs an SVG file that is then imported into Adobe Illustrator or Inkscape for final cartographic styling, labelling, and quality control before publication.

When a network has more than approximately 100 routes, the workflow recommends applying route filtration and abstraction before running LOOM, and validating the filtered GTFS feed before passing it to the plugin. For networks below that threshold the GTFS feed from GIS2GTFS can be passed directly to the plugin as described in Section 4.1.

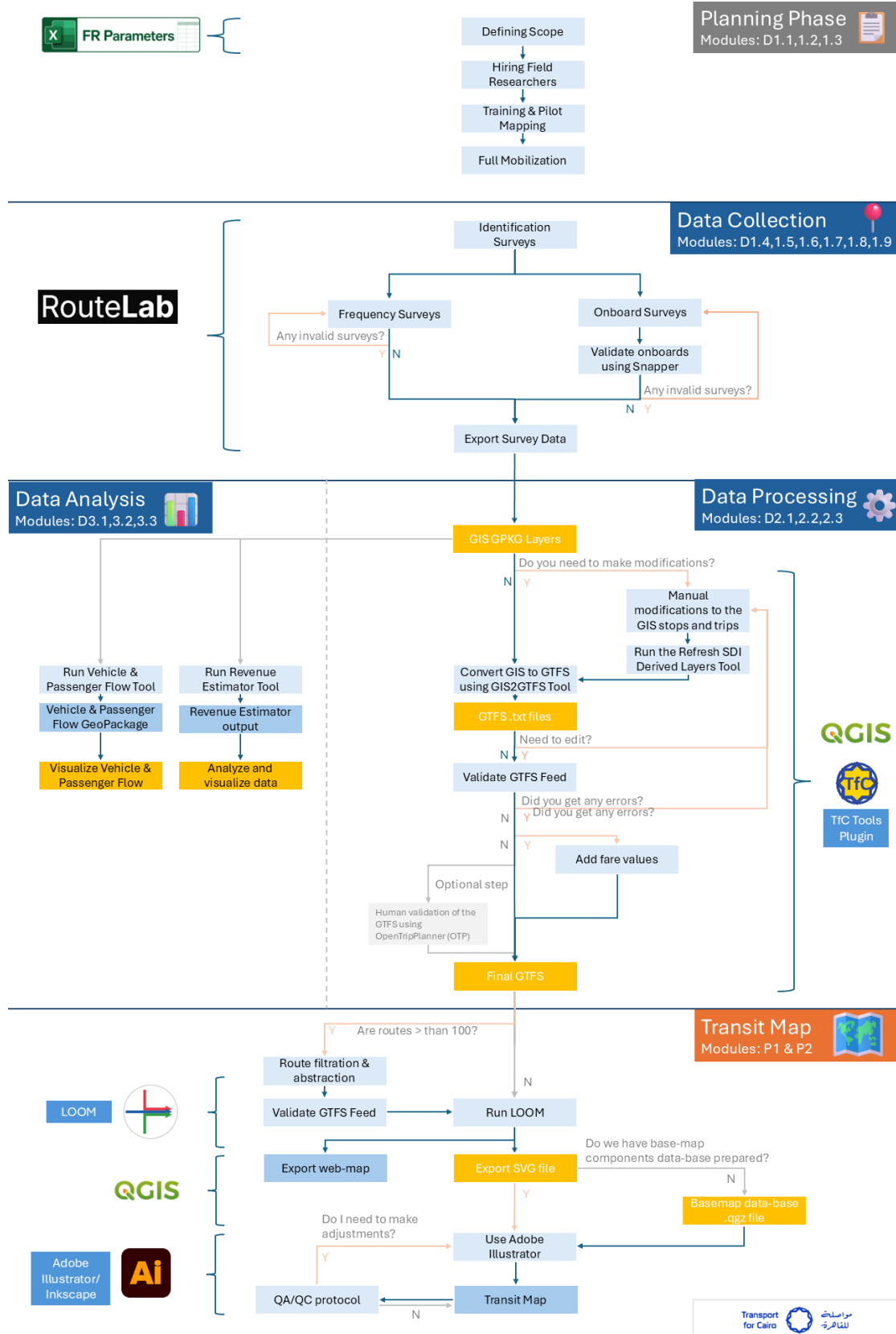


Figure 12 LOOM plugin position in the TfC Napping Workflow (Transit Map phase)

8. Output examples

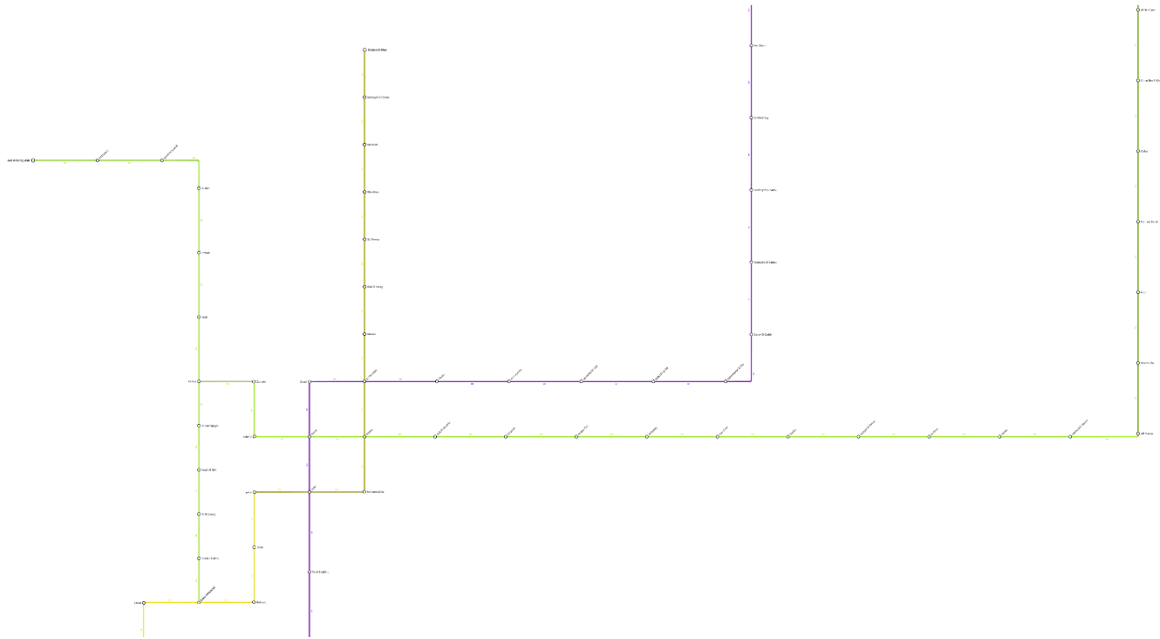


Figure 13 svg output example using GTFS for Cairo Metro lines 1, 2, and 3, ortholinear schematisation, with random colours

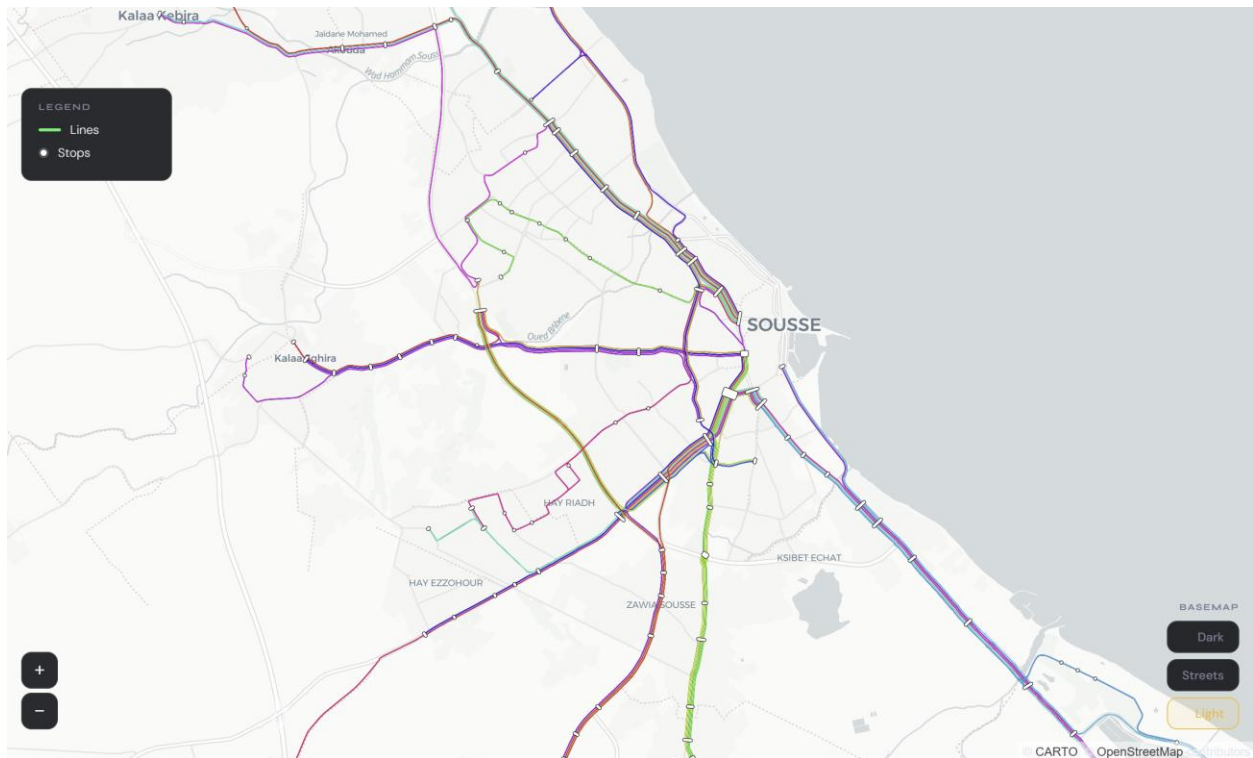


Figure 14 Web map output example using GTFS for public transport in Tunisia, no schematisation, with random colours

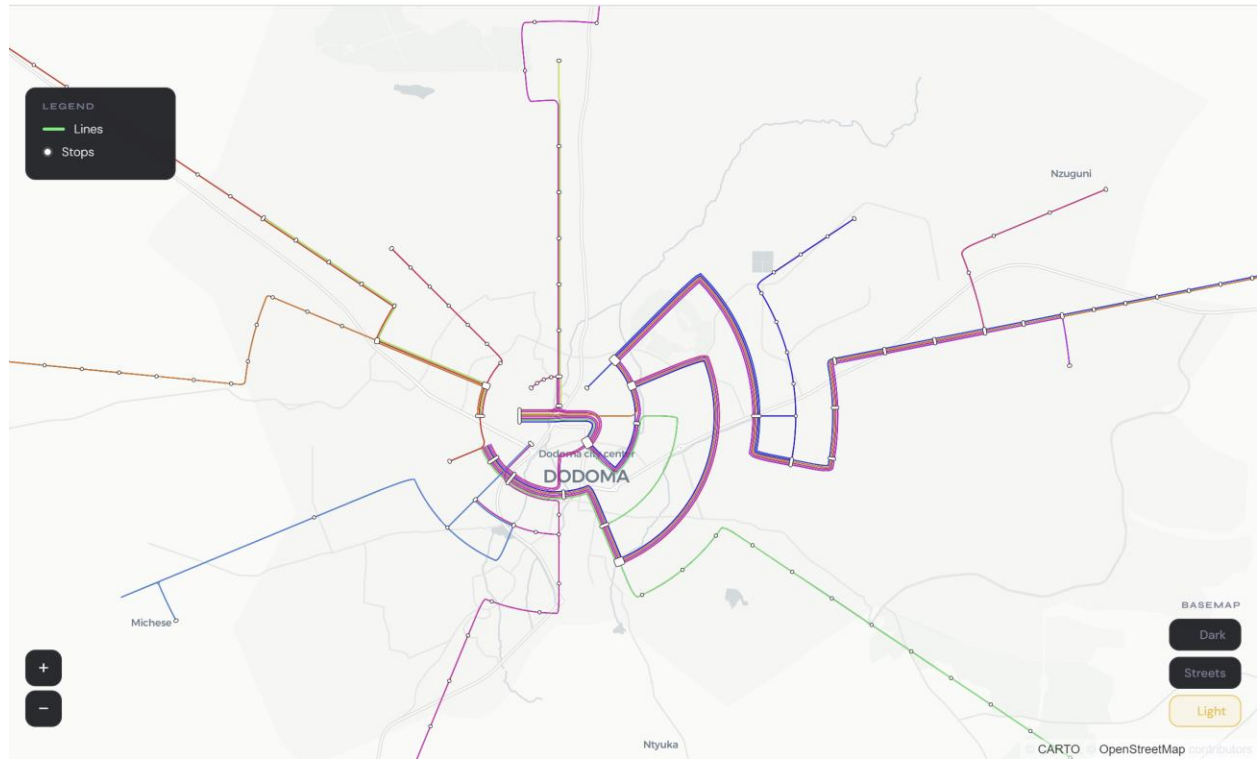


Figure 15 Web map output example using GTFS for popular transport in Tanzania, orthoradial schematisation with random colours



9. External references

1. University of Freiburg's LOOM on Github: <https://github.com/ad-freiburg/loom>
2. Transport for Cairo's LOOM QGIS plugin on Github:
https://github.com/transportforcairo/loom_qgis
3. Transport for Cairo's LOOM QGIS plugin on QGIS: https://plugins.qgis.org/plugins/loom_qgis/
4. Transport for Cairo's Tfc Tools on QGIS: https://plugins.qgis.org/plugins/tfc_tools/
5. Transport for Cairo's Tfc Tools user guide:
https://github.com/transportforcairo/tfc_tools/blob/main/tfc_tools_user_guide.pdf